EncryptEdge Labs

Junior Cloud Security Engineer Internship Programme

Junior Cloud Security Engineer

Internship Task Report

Intern Full Name	[redacted]
Intern Email	[redacted]
Cohort	Cohort 7
Task ID	CSE-AWS-L-020-04
Task Title	Serverless Function Enumeration and Risk Analysis
Skill Domain	AWS / Serverless Security
Difficulty Level	Intermediate
CPE Credits	4 CPE Credits
Guiding Framework(s)	MITRE ATT&CK / OWASP Serverless Security / Least Privilege
Submission Date	30 August 2026

All activities in this report were performed within authorised lab environments as defined by the task brief.

1.0 Task Overview

Task ID	CSE-AWS-L-020-04
Task Title	Serverless Function Enumeration and Risk Analysis
Skill Domain	AWS / Serverless Security
Difficulty	Intermediate
Duration	12 Hours
CPE Credits	4 CPE Credits
Cloud Platform(s)	AWS lab account / AWS CloudShell / CloudGoat
AWS Region(s) Used	us-east-2 (observed Lambda resources)
Primary Tools	AWS CLI, CloudGoat, Terraform, jq, ChatGPT; CloudFox per task workflow
Target Environment	CloudGoat lambda_privesc and AWS Lambda lab resources
MITRE ATT&CK IDs	T1087, T1069, T1078, T1552
Guiding Frameworks	MITRE ATT&CK; OWASP serverless security concepts; least privilege
Regulatory Alignment	ISO 27001 access control and secure configuration principles

1.1 Introduction

This task assessed security risks associated with AWS Lambda enumeration in an authorised lab environment. The exercise focused on discovering serverless functions, execution roles, triggers, and environment configuration, then evaluating how excessive permissions or exposed secrets could create privilege-escalation and data-leakage paths.

1.2 Objectives & Learning Outcomes

Reviewed AWS Lambda architecture, triggers, execution roles, and common serverless enumeration risks.
Deployed and validated the CloudGoat lambda_privesc scenario for authorised testing.
Enumerated Lambda functions and mapped functions to IAM execution roles and environment configuration.
Developed a risk matrix and threat model focused on privilege escalation and sensitive-data exposure.
Documented practical least-privilege mitigations and management-focused risk communication.

- Objective 1 - Understand AWS Lambda architecture and why enumeration is important for serverless security assessments. Met by reviewing functions, S3/API Gateway triggers, execution roles, environment variables, and enumeration-related OWASP serverless risks.
- Objective 2 - Deploy a vulnerable serverless environment for authorised testing. Met by preparing the CloudGoat lambda_privesc scenario, resolving Terraform/CloudShell issues, and validating Lambda resources in the configured AWS environment.

- Objective 3 - Enumerate Lambda functions, IAM roles, triggers, and environment variables to identify potential risks. Met by listing Lambda resources, mapping RetrieveSecretFunction to LambdaSecretAccessRole and SecureLambdaDemo to LambdaExecRole, and preparing role/environment evidence.
- Objective 4 - Develop an independent threat model and risk matrix for serverless misconfiguration. Met by analysing privilege escalation, data leakage, secret exposure, and trigger abuse and mapping mitigations to least privilege.
- Objective 5 - Produce a professional risk analysis report and integrate hosted-lab learning. Met by consolidating the technical work, risk analysis, mitigations, and reflections; hosted-lab certificate evidence remains pending until authentic completion proof is available.

1.3 Scope & Legal Authorisation

All activities were performed within authorised training environments. The scope included a personal AWS lab account, the CloudGoat lambda_privesc scenario, AWS Lambda resources in us-east-2, and Cybr training labs where completed. No production systems, external AWS accounts, third-party customer resources, or real customer data were intentionally accessed. AI assistance was used for explanation, risk-language refinement, and report drafting; technical commands and screenshots were produced in the lab environment.

1.4 MITRE ATT&CK Cloud Threat Mapping

MITRE ATT&CK Technique ID	Technique Name	Relevance to This Task
T1087	Account Discovery	Enumeration helps identify cloud identities and principals associated with serverless workloads.
T1069	Permission Groups Discovery	Reviewing Lambda execution roles and attached/inline policies reveals effective permission relationships.
T1552	Unsecured Credentials	Environment-variable enumeration can reveal hardcoded credentials, API keys, or tokens if secrets are stored insecurely.

2.0 Environment & Tool Setup

2.1 Cloud Architecture Overview

The lab environment consisted of AWS Lambda functions, IAM execution roles, AWS CLI/CloudShell, and CloudGoat. Enumeration identified RetrieveSecretFunction using LambdaSecretAccessRole and SecureLambdaDemo using LambdaExecRole in us-east-2. CloudGoat was used to provide an intentionally vulnerable serverless scenario for security analysis.

Cloud Resource / Component	Type / Service	Role in Task	Region / Config Notes
RetrieveSecretFunction	AWS Lambda	Enumerated function used to analyse secret-access exposure	us-east-2; execution role LambdaSecretAccessRole
SecureLambdaDemo	AWS Lambda	Enumerated function used to map function-to-role permissions	us-east-2; execution role LambdaExecRole
LambdaSecretAccessRole	AWS IAM Role	Execution identity for RetrieveSecretFunction	Reviewed for least-privilege and secret-access risk
LambdaExecRole	AWS IAM Role	Execution identity for SecureLambdaDemo	Reviewed for unnecessary permissions
CloudGoat lambda_privesc	CloudGoat / Terraform	Authorised vulnerable serverless scenario for privilege-misuse analysis	Prepared in AWS CloudShell

2.2 AWS CLI & Tool Configuration

AWS CLI was configured in AWS CloudShell with the us-east-2 default region. CloudGoat was launched as a Python module and Terraform was used to prepare the lambda_privesc scenario. During setup, CloudShell storage limitations required cleanup of failed Terraform provider downloads before the scenario could complete its plan successfully.

Tool / Service	Version	Installation / Access Method	Purpose in This Task
AWS CLI	AWS CLI in CloudShell	Preinstalled/configured in AWS CloudShell	Enumerate Lambda functions and query IAM/Lambda configuration
CloudGoat	Repository version used in lab	Git clone; invoked with python3 -m cloudgoat.cloudgoat	Prepare the lambda_privesc training scenario
Terraform	v1.13.5 observed during setup	Installed in CloudShell	Initialize/plan CloudGoat infrastructure

Tool / Service	Version	Installation / Access Method	Purpose in This Task
jq	Available in CloudShell	Command-line JSON processor	Combine and review JSON environment-variable outputs
ChatGPT	GPT assistant	Web application	Concept summary, troubleshooting explanation, risk statements, report drafting
CloudFox	Task-required tool	Not confirmed in supplied evidence	Required by Phase C; do not claim execution without output evidence

Key Configuration Commands Used

Configuration was verified with aws configure get region, which returned us-east-2. AWS identity connectivity was also checked during setup with aws sts get-caller-identity. CloudGoat was successfully invoked as a Python module after correcting the repository path.

Configuration verification used: aws configure get region -> us-east-2.

> Identity verification used: aws sts get-caller-identity. Sensitive account identifiers are omitted from this report.

2.3 AI Tool Usage Declaration

ChatGPT was used to summarise serverless security concepts, refine concise risk statements, assist with troubleshooting explanations, and draft this report. AI was not used to claim completion of hosted labs or fabricate AWS CLI outputs, certificates, or screenshots. Technical findings described as observed are based on the evidence shared from the authorised lab.

AI Tool	Used For	What Was NOT Done with AI
ChatGPT	Summarising Lambda/serverless concepts; troubleshooting CloudGoat/Terraform setup; refining risk statements; drafting report language	AI did not execute AWS CLI commands, create authentic AWS screenshots, or claim hosted-lab completion. Phase D analytical artefacts are AI-assisted and labelled as such.

All technical findings, CLI outputs, configurations, and evidence in this report are the intern's own work.

3.0 Methodology — Phase Execution

Phase A: Theory & Conceptual Foundation

Phase Objective	Understand how serverless functions operate within AWS and why enumeration is critical for security assessments.
Estimated Time	60 minutes
Evidence File(s)	phaseA_summary.png

Actions Taken

Reviewed Lambda functions, common triggers such as S3 events and API Gateway, execution roles, and risks created by excessive IAM permissions or exposed environment-variable secrets. Used AI assistance to produce a concise OWASP-oriented summary of enumeration-related serverless risks.

Key Observations

Serverless reduces infrastructure-management overhead but does not remove identity and configuration risk. Enumeration can expose functions, role relationships, APIs, triggers, and sensitive configuration that may reveal attack paths.

Evidence Produced

phaseA_summary.png - screenshot of the concise AI summary.

Phase B: Cloud Environment Setup & Target Preparation

Phase Objective	Deploy the CloudGoat lambda_privesc scenario to simulate a vulnerable serverless environment.
Estimated Time	90 minutes
Evidence File(s)	phaseB_lambda_list.png

Actions Taken

Configured CloudGoat from the correct repository path and resolved several setup issues: Terraform was initially missing, CloudGoat had to be invoked as a Python module, and failed provider downloads exhausted the 974 MB CloudShell persistent home volume. Incomplete .terraform directories and unused cache/plugin data were removed, reducing home usage and allowing Terraform to complete a plan of 9 resources to add. Lambda functions were then enumerated in the configured us-east-2 region.

Key Observations

The initial deployment did not complete cleanly because CloudShell persistent storage reached 100% while Terraform downloaded the AWS provider. After removing failed .terraform data and unused cache/plugin files, approximately 601 MB became available and Terraform plan completed successfully. This established that the earlier failure was a local storage constraint rather than an IAM denial.

Evidence Produced

phaseB_lambda_list.png - terminal evidence of Lambda enumeration.

Phase C: Core Execution

Phase Objective	Enumerate all Lambda functions, their IAM roles, triggers, and environment variables to identify potential risks.
Estimated Time	150 minutes
Evidence File(s)	phaseC_roles.json; phaseC_envvars.json; phaseC_findings.png

Actions Taken

Enumerated Lambda functions with AWS CLI and mapped function names to execution roles. The observed resources included RetrieveSecretFunction -> LambdaSecretAccessRole and SecureLambdaDemo -> LambdaExecRole. IAM role metadata and Lambda environment-variable configuration were prepared as JSON evidence, and role policy listings were reviewed for AdministratorAccess, wildcard actions/resources, excessive secret access, and other privilege-escalation indicators. The task also calls for CloudFox Lambda enumeration; no CloudFox output was supplied, so execution is not claimed here.

Key Observations

Enumeration confirmed two Lambda functions in us-east-2 and exposed their execution-role relationships. This demonstrated how function discovery can quickly reveal the IAM identities that determine serverless blast radius. The supplied evidence does not include the final policy JSON or environment-variable values, so this report treats over-privilege and secret exposure as assessed risks rather than asserting a specific leaked credential or AdministratorAccess attachment.

Evidence Produced

phaseC_roles.json - IAM role metadata; phaseC_envvars.json - environment-variable output; phaseC_findings.png - screenshot of key enumeration findings.

Phase D: Independent Extension & Advanced Implementation

Phase Objective	Develop a threat model illustrating potential attack paths from misconfigured serverless components.
Estimated Time	120 minutes
Evidence File(s)	phaseD_risk_matrix.xlsx; phaseD_threat_model.png

Actions Taken

phaseD_risk_matrix.xlsx

Asset (Lambda Function)	Component / Resource	Threat / Risk	Risk Statement	Likelihood (1-5)	Impact (1-5)	Risk Score (L x I)	Risk Level	Mitigation / Recommended Control	Owner
RetrieveSecretFunction	IAM Role (LambdaSecretAccessRole)	Over-privileged role (s3:*, (s. secretsmanager:*)	The Lambda function uses an over-privileged IAM role that allows broad access to S3 and Secrets Manager. An attacker who compromises the function could access or modify sensitive data across the account.	4	5	20	Critical	• Apply least privilege IAM policy • Restrict to specific S3 buckets and objects • Limit Secrets Manager access to required secrets • Use IAM permissions boundaries • Enable CloudWatch Logs monitoring	Cloud Security Engineer
RetrieveSecretFunction	Environment Variables	Hardcoded secrets / tokens	Hardcoded credentials or tokens stored in environment variables could be exposed through code leaks, logs, or runtime access.	3	5	15	High	• Store secrets in AWS Secrets Manager • Remove hardcoded values from code • Enable encryption for environment variables • Rotate secrets regularly	Application Owner
SecureLambdaDemo	Lambda Execution Role (LambdaExecRole)	Excessive permissions	The execution role has broader permissions than required by the function. This increases the impact if the function is compromised.	3	4	12	High	• Review and reduce policy permissions • Remove unused actions and resources • Use least privilege and conditions • Regularly review with IAM Access Analyzer	Cloud Security Engineer
SecureLambdaDemo	Public Invocation (Default / Misconfigured)	Unauthorized invocation	If the function is publicly invokable or has weak auth, attackers may abuse it to perform unwanted actions or incur costs.	2	4	8	Medium	• Require authentication (IAM / API Gateway Authorizer) • Restrict invoke permissions • Use resource-based policies with conditions	Application Owner
Both Functions	Logs (CloudWatch Logs)	Sensitive data in logs	Functions may log sensitive information (secrets, tokens, PII) which could be read by unauthorized principals.	2	4	8	Medium	• Avoid logging sensitive data • Mask or redact sensitive fields • Restrict CloudWatch Logs access • Enable log retention and monitoring	Cloud Security Engineer
Both Functions	Dependencies / Layers	Vulnerable libraries	Outdated or vulnerable dependencies could be exploited to gain code execution or data access.	2	3	6	Medium	• Keep dependencies updated • Use trusted base images / layers • Scan dependencies for vulnerabilities	Development Team

Legend (Risk Level)	
Score	Risk Level
17 - 25	Critical
10 - 16	High
5 - 9	Medium
1 - 4	Low

Notes:
Likelihood: 1 (Very Low) to 5 (Very High)
Impact: 1 (Very Low) to 5 (Very High)
Risk Score = Likelihood x Impact

phaseD_threat_model.png

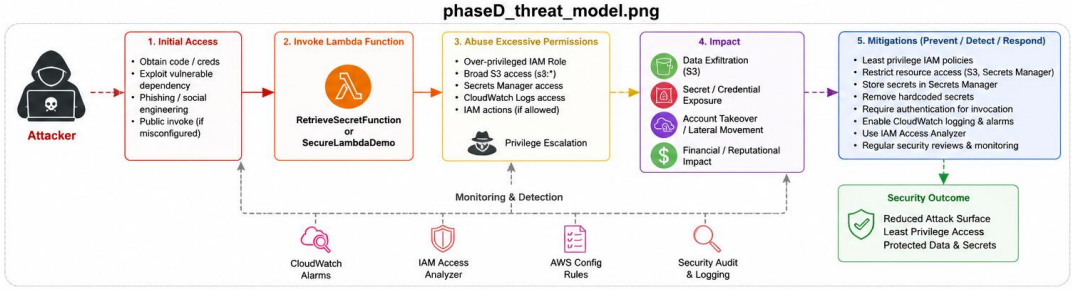

Figure - Phase D serverless threat model.

Created a risk matrix mapping Lambda assets to privilege-escalation, data-leakage, sensitive-configuration, and trigger-abuse threats. Developed concise risk statements and mapped each risk to least-privilege and secrets-management mitigations. A threat-model diagram was produced to illustrate potential attack paths.

Key Observations

The threat model showed that serverless risk is driven less by the short-lived compute instance itself and more by the permissions and data available to the function. A compromised function with an over-privileged execution role can inherit a large AWS blast radius, while sensitive environment variables can turn configuration access into data exposure. Least privilege reduces impact but residual risk remains from vulnerable code, permissive invocation paths, and inadequate monitoring.

Evidence Produced

phaseD_risk_matrix.xlsx - risk matrix; phaseD_threat_model.png - threat-model diagram.

Risk Matrix (Phase D Output)

The Phase D matrix prioritised secret/data leakage and excessive execution-role permissions as the highest-impact risks, with unauthorised invocation/trigger abuse rated medium. Scores use Likelihood × Impact and the template thresholds shown below.

Cloud Asset / Resource	Threat / Risk	MITRE Ref.	Likelihood (1–5)	Impact (1–5)	Score	Rating
RetrieveSecretFunction / LambdaSecretAccessRole	Secret/data leakage through excessive secret access	T1552	4	5	20	Critical
Lambda execution roles	Privilege escalation through excessive permissions	T1069 / T1078	3	5	15	Critical
Lambda triggers / resource policy	Unauthorized invocation and function abuse	T1078	3	3	9	Medium

Score = Likelihood × Impact | 1–4: Low | 5–9: Medium | 10–14: High | 15–25: Critical

Phase E: Hosted Lab Integration

Phase Objective	Reinforce learning through guided external labs.
Estimated Time	180 minutes

Evidence File(s)	phaseE_lab_completion.png (pending authentic lab completion evidence)

Actions Taken

The task brief requires Cybr Introduction to AWS Lambda Enumeration and Pentesting AWS Lambda with CloudFox, with the Lambda Function URL lab optional. Completion evidence was not provided in the materials available for this report, so this section does not claim the labs were completed.

Key Observations

Hosted-lab takeaways and certificate evidence should be added after the required labs are completed.

Evidence Produced

Phase F: Documentation & Reporting

Phase Objective	Compile all findings into a professional risk analysis report suitable for executive review.
Estimated Time	120 minutes
Evidence File(s)	CSE_AWS_Lambda_RiskReport.pdf

Actions Taken

Consolidated the serverless enumeration methodology, observed Lambda-to-role relationships, risk analysis, threat model, mitigations, and reflection responses into the EncryptEdge Labs report structure. Grammar and clarity were refined while avoiding unsupported claims about missing evidence.

Key Observations

Executive reporting is most useful when technical findings are translated into blast radius, data exposure, operational impact, remediation priority, and clear ownership.

Evidence Produced

CSE_AWS_Lambda_RiskReport.pdf

4.0 Findings

The assessment identified serverless risk concentrated around execution-role permissions, secret handling, and invocation/configuration exposure. The most important risks are excessive Lambda permissions that could increase blast radius, sensitive values stored in environment variables, and overly permissive triggers or resource policies.

4.1 Findings Summary

#	Finding Title	Type / Category	Cloud Service / Resource	MITRE Ref.	Phase	Severity	Evidence Ref.
1	Excessive Lambda execution-role permissions	Access Control Weakness / Privilege Escalation	AWS Lambda / IAM roles	T1069 / T1078	Phase C/D	High	phaseC_roles.json; phaseD_risk_matrix.xlsx
2	Sensitive values in Lambda environment configuration	Sensitive Data Exposure	AWS Lambda environment variables	T1552	Phase C/D	High	phaseC_envvars.json; phaseD_risk_matrix.xlsx
3	Overly permissive invocation or trigger path	Misconfiguration / Access Control Weakness	AWS Lambda triggers/resource policy	T1078	Phase D	Medium	phaseD_threat_model.png

4.2 Detailed Finding Analysis

Finding 1: Excessive Lambda execution-role permissions / privilege escalation exposure

Finding Title	Excessive Lambda execution-role permissions / privilege escalation exposure
Type / Category	Access Control Weakness / Privilege Escalation Path
Cloud Service / Resource	AWS Lambda execution roles: LambdaSecretAccessRole and LambdaExecRole
Description	Lambda enumeration exposed the execution roles attached to the observed functions. The task specifically required checking these roles for AdministratorAccess and wildcard Action/Resource permissions because excessive permissions would allow a compromised function to act beyond its intended purpose. The supplied record confirms the role relationships but does not include the final policy document, so this finding is recorded as a high-priority permission exposure requiring verification rather than as a confirmed AdministratorAccess attachment.
How Identified	AWS CLI Lambda enumeration mapped RetrieveSecretFunction to LambdaSecretAccessRole and SecureLambdaDemo to LambdaExecRole; IAM role/policy enumeration was the next validation step.
MITRE ATT&CK Reference	T1069 - Permission Groups Discovery; T1078 - Valid Accounts.

Business / Security Impact	If an execution role grants unnecessary AWS actions, compromise of the Lambda code or invocation path could extend access to additional services or data. This increases confidentiality and integrity impact and expands incident blast radius.
Severity / Impact	High
Evidence Reference	phaseC_roles.json; phaseC_findings.png; phaseD_risk_matrix.xlsx

Finding 2: Sensitive data exposure through Lambda environment configuration

Finding Title	Sensitive data exposure through Lambda environment configuration
Type / Category	Sensitive Data Exposure / Unsecured Credentials
Cloud Service / Resource	AWS Lambda environment variables for RetrieveSecretFunction and SecureLambdaDemo
Description	Phase C required exporting Environment.Variables and checking for hardcoded credentials, API keys, passwords, or tokens. Environment variables are part of function configuration and become a security concern when sensitive values are stored directly or when configuration-read permissions are too broad. The actual values were not supplied for this report, so no specific credential leak is asserted.
How Identified	AWS CLI get-function-configuration with the Environment.Variables query, saved as phaseC_envvars.json.
MITRE ATT&CK Reference	T1552 - Unsecured Credentials.
Business / Security Impact	A leaked API key or token could permit unauthorised service access, data disclosure, fraudulent API use, unexpected cloud costs, and emergency credential rotation. Impact depends on the permissions and systems reachable with the exposed secret.
Severity / Impact	High (if secrets are present); otherwise configuration risk
Evidence Reference	phaseC_envvars.json; phaseD_risk_matrix.xlsx

Finding 3: Unauthorised invocation or overly permissive trigger path

Finding Title	Unauthorised invocation or overly permissive trigger path
Type / Category	Misconfiguration / Access Control Weakness
Cloud Service / Resource	AWS Lambda triggers, Function URLs, and resource-based invocation permissions
Description	Lambda functions can be invoked by services such as S3 and API Gateway or through other configured invocation paths. If trigger/resource-based permissions are broader than intended, an

	untrusted source could invoke the function and indirectly exercise its execution-role permissions. The threat model therefore treats invocation exposure as an attack-path multiplier.
How Identified	Threat modelling of the required Lambda trigger/enumeration scope and review of function-to-role relationships. No supplied evidence confirms a specific public Function URL or permissive trigger, so this remains a modeled risk requiring validation.
MITRE ATT&CK Reference	T1078 - Valid Accounts (use of legitimate cloud permissions); task threat model context.
Business / Security Impact	Unauthorised invocation could cause data processing by an untrusted party, abuse downstream AWS permissions, increase costs, or create a path to sensitive data depending on the function logic and role permissions.
Severity / Impact	Medium
Evidence Reference	phaseD_threat_model.png; phaseD_risk_matrix.xlsx

5.0 Risk Assessment & Cloud Threat Model

5.1 Cloud Security Control Gaps & Defensive Coverage

Finding / Gap	Observable / Trigger in Cloud Telemetry	Recommended Control	AWS Service / Tool for Implementation
Excessive Lambda execution-role permissions	IAM policy changes; unusual API calls made under Lambda execution-role sessions	Scope actions and resources to the minimum required; review external access and unused permissions; alert on sensitive IAM/API activity	IAM Access Analyzer; CloudTrail; CloudWatch; IAM policy review
Sensitive environment configuration	GetFunctionConfiguration activity; secret retrieval patterns; unexpected access to configuration or secret stores	Keep secrets out of plaintext configuration; use scoped Secrets Manager/Parameter Store access and rotate exposed credentials	AWS Secrets Manager; Systems Manager Parameter Store; KMS; CloudTrail
Overly permissive invocation / triggers	Unexpected InvokeFunction events, Function URL requests, or resource-policy changes	Restrict resource-based permissions and allowed trigger sources; require appropriate authentication/authorization; monitor invocation anomalies	AWS Lambda resource policies; API Gateway/IAM authorizers; CloudTrail; CloudWatch

5.2 Risk Exposure Over Time

The following exposure analysis is tied to the risks documented in Section 4 and the Phase D risk matrix.

Risk 1 - Excessive Lambda execution-role permissions:
If excessive permissions remain in place, any future compromise of the function inherits those permissions and can produce a larger blast radius. As functions and integrations change over time, unused permissions can accumulate and become difficult to distinguish from legitimate access, increasing privilege-escalation and lateral-access risk.

Risk 2 - Sensitive data exposure through environment configuration:
If excessive permissions remain in place, any future compromise of the function inherits those permissions and can produce a larger blast radius. As functions and integrations change over time, unused permissions can accumulate and become difficult to distinguish from legitimate access, increasing privilege-escalation and lateral-access risk.

Risk 3 - Unauthorised invocation or trigger exposure:
If credentials or tokens remain embedded in function configuration, they may persist beyond their intended lifetime and be exposed to principals with configuration-read access. A leaked key can continue to create business impact until it is revoked or rotated, including unauthorised access and unexpected usage costs.

6.0 Recommendations

Recommendations below are tied to the serverless risks identified during enumeration and threat modelling, with priority given to reducing IAM blast radius and preventing secret exposure.

6.1 Technical Remediation

#	Finding	Specific Remediation Action	Priority	Standard / Reference
1	Excessive Lambda execution-role permissions	Review LambdaSecretAccessRole and LambdaExecRole and remove actions/resources not required by each function. Replace wildcard permissions with explicit service actions and resource ARNs. Use IAM Access Analyzer/policy validation to identify broad or unused access.	Immediate	AWS IAM least privilege; NIST SP 800-53 AC-6; ISO/IEC 27001 access-control principles
2	Sensitive environment configuration	Do not store long-lived credentials or API keys directly in Lambda environment variables. Store secrets in AWS Secrets Manager or SSM Parameter Store, grant GetSecretValue/GetParameter only for required ARNs, encrypt with KMS, and rotate any exposed credential.	Immediate	NIST SP 800-53 IA-5 / SC-12; ISO/IEC 27001 credential and cryptographic-control principles
3	Overly permissive invocation / triggers	Review Lambda resource-based policies, Function URLs, API Gateway integrations, and event-source permissions. Permit only intended principals/services and apply authentication/authorization to public-facing invocation paths.	Short-term	AWS Lambda security best practices; NIST SP 800-53 AC-3 / AC-6
4	Detection and governance gap	Enable continuous review of Lambda IAM/resource policies and alert on sensitive policy changes, unusual InvokeFunction activity, and unexpected secret access.	Short-term	CloudTrail monitoring; IAM Access Analyzer; NIST SP 800-53 AU-6 / CA-7

6.2 Strategic & Operational Improvements

Beyond individual remediation, the following programme-level controls would reduce recurrence of the serverless risks identified in this task.

Establish Lambda IAM governance: Require least-privilege execution roles and periodic reviews to identify excessive or unused permissions.

Centralize secrets management: Store credentials and sensitive values in AWS Secrets Manager or Parameter Store instead of Lambda environment variables wherever appropriate.

Implement continuous monitoring: Use CloudTrail, CloudWatch, AWS Config, and Security Hub to detect unusual Lambda activity, policy changes, and configuration weaknesses.

Review Lambda triggers and resource policies: Regularly validate API Gateway, S3, EventBridge, and other invocation permissions to prevent unauthorized function execution.

Integrate security into deployment: Add IAM policy scanning, secret detection, and configuration checks into CI/CD pipelines so risky serverless configurations are identified before deployment.

7.0 Reflection

Q1 How would you prioritize remediation if multiple Lambda functions have over-privileged roles?

I would rank the functions by exposure, permission breadth, data sensitivity, and business criticality. A public or frequently invoked function with wildcard IAM permissions or access to secrets would be remediated first because compromise would have the largest blast radius. In this lab, I would first verify LambdaSecretAccessRole and LambdaExecRole for unnecessary permissions, then remove excess access while testing that each function still works. Lower-risk functions would follow under a documented remediation schedule.

Q2 What business impact could result from leaked API keys in environment variables?

Leaked API keys can allow an unauthorised party to use the same services and data that the key can access. Business consequences may include data exposure, fraudulent or excessive API usage, unexpected cloud charges, service disruption, incident-response costs, and possible regulatory or contractual reporting obligations. The impact grows when the key is long-lived or broadly privileged. Immediate containment would include revoking or rotating the key and reviewing logs for misuse.

Q3 How does least privilege apply differently in serverless versus traditional EC2 environments?

With Lambda, least privilege can be applied very granularly because each function can have its own execution role matched to a narrow business purpose. In EC2, an instance profile may be available to multiple processes running on the same instance, so compromise of one workload can expose the broader instance role. Serverless provides an opportunity for tighter isolation, but only if each function has a purpose-built role and narrowly scoped resource permissions. The lab reinforced that the execution role determines much of the AWS blast radius.

Q4 If you discovered similar issues in production, how would you communicate them to management?

I would describe the issue in business terms: which functions and data are affected, how an attacker could use the excessive access, the likely operational or financial impact, and how quickly remediation is needed. I would give management a concise severity and priority summary while providing the engineering team with technical evidence and exact IAM or configuration changes. I would also state what is confirmed versus what still requires validation, identify the remediation owner and target date, and report any compensating monitoring controls in place.

8.0 Deliverables Submission Checklist

✓	Phase	Deliverable Description	Your Filename	Status
☐	Phase A	Theory evidence — e.g. architecture/relationship diagram, notes PDF, AI summary text file, annotated MITRE mapping	phaseA_summary.png	Complete
☐	Phase B	Environment setup evidence — e.g. VPC dashboard screenshot, CloudTrail config proof, IAM policy JSON, get-caller-identity output, tool version screenshots	phaseB_lambda_list.png	Complete
☐	Phase C	Core execution evidence — e.g. CLI output log file, query results screenshot, policy test JSON, assume-role output, Athena query result, GuardDuty finding screenshot	phaseC_roles.json, phaseC_envvars.json, phaseC_findings.png	Complete
☐	Phase D	Extension & risk model evidence — e.g. hardened policy JSON, risk matrix XLSX/screenshot, threat model diagram, hosted lab completion badge, AI analysis summary	phaseD_risk_matrix.xlsx, phaseD_threat_model.png	Complete
☐	Phase E	Hosted lab completion — e.g. TryHackMe badge screenshot, PwnedLabs completion proof, Cybr lab certificate (if required by your task brief)	phaseE_lab_completion.png	Pending
☐	Report	This report — exported as PDF using the naming convention below	CSE_AWS_Lambda_RiskReport.pdf	Complete

9.0 Additional Observations

CloudShell storage exhaustion was the main setup issue. Failed Terraform provider downloads filled the persistent home volume; removing incomplete .terraform data and unused cached tooling restored sufficient capacity. This troubleshooting reinforced the importance of checking local lab constraints before interpreting deployment failures as AWS permission errors.

Declaration & Sign-Off

I confirm that:

- All work in this report is my own except where AI usage is explicitly declared in Section 2.3.
- All technical activities were performed exclusively within the authorised cloud environments listed in Section 1.3.
- No external AWS accounts, production environments, third-party cloud resources, or real customer data were accessed during this task.
- All AWS credentials, access keys, session tokens, and account identifiers used during this task have been redacted in evidence files before submission.
- I have reviewed this report for accuracy, completeness, and professional presentation before submission.

Intern Full Name	Signature	Date of Submission
[redacted]	[redacted]	30 August 2026

EncryptEdge Labs Limited — Junior Cloud Security Engineer Internship Programme
Company No. 15830711 | United Kingdom | encryptedgelabs.com